Day 4 - LLM Powered Ticket Classification (Structured Output)

Objective

Build an LLM-based pipeline that reads customer support tickets and predicts Category, Urgency, and Sentiment in a strict JSON schema. The pipeline should validate responses, compare prompt strategies, and classify all 45 tickets.

Technologies Used

Python, Groq API, Llama 3.3 70B Versatile, CSV, JSON, python-dotenv.

Tasks Completed

• Integrated Groq LLM API.
• Implemented Zero-shot prompting.
• Implemented Few-shot prompting with three examples.
• Added schema validation and JSON parsing.
• Processed all 45 support tickets.
• Generated CSV and JSON outputs.
• Compared prompt performance using 14 labeled tickets.
• Prepared LLM_REPORT.md and Git repository.

Prompt Strategies

Zero-shot: Direct instructions with required JSON schema.

Few-shot: Added three labeled examples before asking the model to classify the ticket.

Validation Logic

1. Parse model response as JSON.
2. Verify required fields exist.
3. Ensure Category, Urgency and Sentiment belong to allowed values.
4. Reject or log invalid responses.

Accuracy Comparison

Field	Zero-shot	Few-shot
Category	13/14 (92.86%)	12/14 (85.71%)
Urgency	9/14 (64.29%)	9/14 (64.29%)
Sentiment	13/14 (92.86%)	13/14 (92.86%)

Winning Strategy

Zero-shot prompting was selected because it achieved higher Category accuracy while matching the Few-shot approach for Urgency and Sentiment.

Output Files

- ticket_classifications.csv
- ticket_classifications.json
- ticket_classifications_few_shot.csv
- ticket_classifications_few_shot.json
- LLM_REPORT.md

Git Summary

Initialized a Git repository, created a .gitignore file, committed the project successfully, and prepared it for GitHub upload.

Conclusion

The project successfully demonstrates structured output generation using an LLM. The pipeline validates responses, compares prompt strategies using labeled data, and produces reusable structured outputs for support ticket triage.